ОТЧЁТ ПО ЗАДАНИЮ 3

MapReduce и DuckDB

Выполнил: Муртазалиев М. А.

Группа J3310

ИСУ 466797

2026

1 Развёртывание RustFS

Я запустил RustFS 1.0.1 с четырьмя каталогами disk1-disk4 и схемой 2+2. Использовал бинарный файл из второго дз, а для третьего создал отдельные каталоги. S3 - 127.0.0.1:9100, консоль -127.0.0.1:9101.

Доп информацию можно в readme найти.

Консоль показывает один сервер и четыре диска в сети.

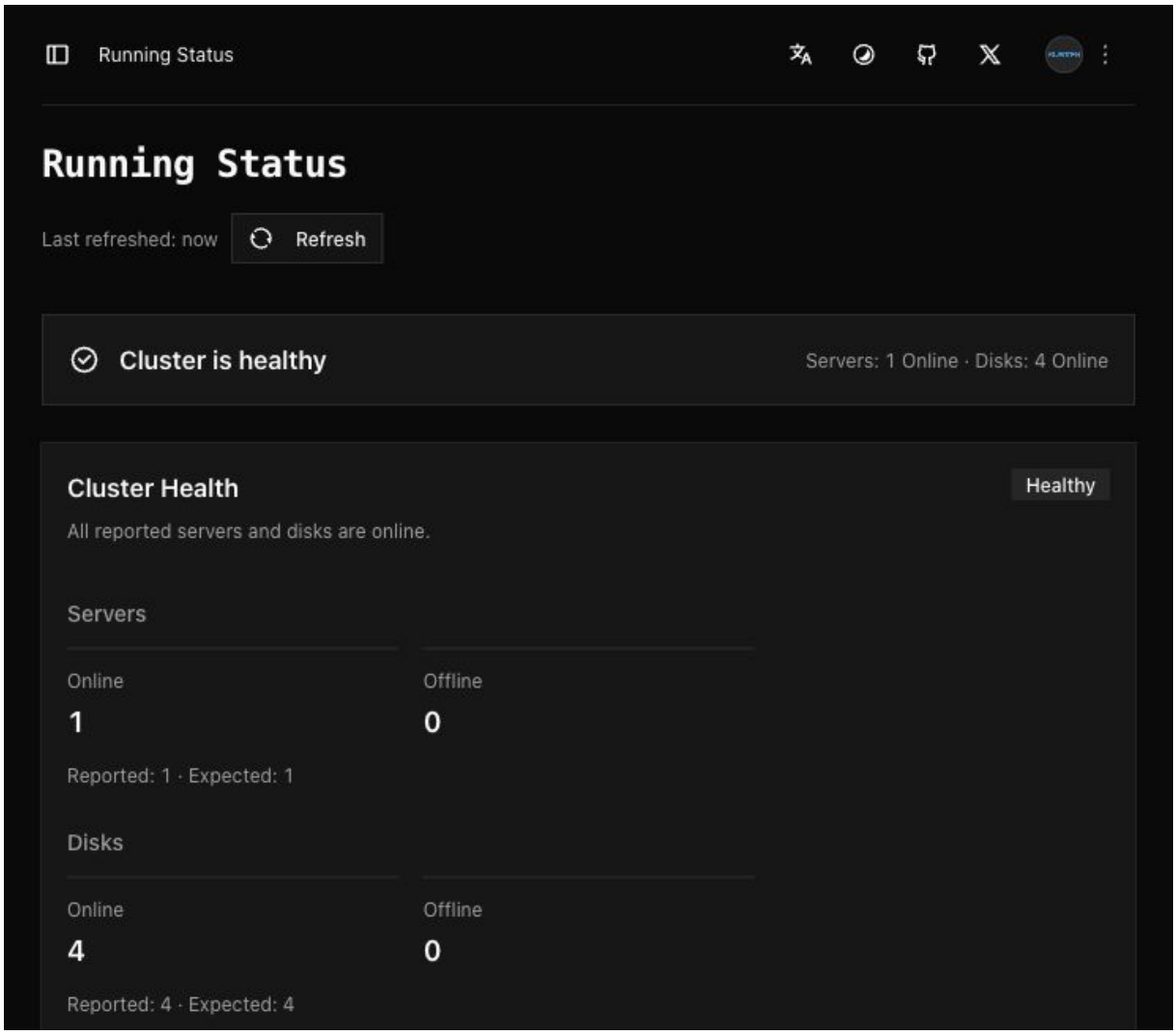


Рисунок 1 - Состояние сервера и четырёх дисков RustFS

2 Загрузка данных

У меня вариант 9 - **«Программист, разработчик на мобильных платформах».**

Нужный csv я сжал и в бакет загрузил.

```
python3 scripts/upload.py
"data_uri":
  "s3://hh-lake/raw/it_vacancies_2024.csv.gz",
"http": 200,
"original_bytes": 847619384,
"gzip_bytes": 295480524
```

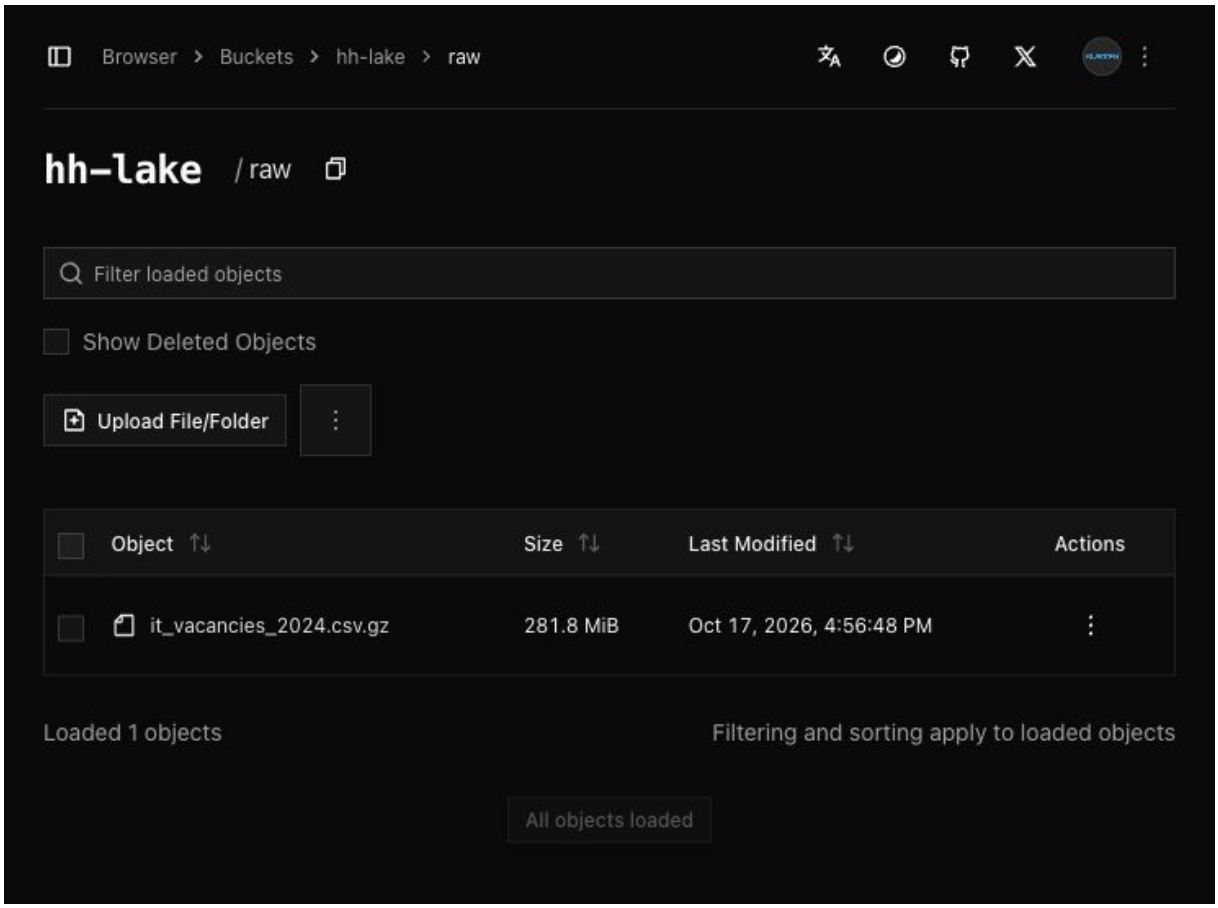


Рисунок 2 - Исходный объект в бакете hh-lake

3 Подсчёт навыков

Получил **3640** вакансий, из них **2709** с навыками. Навыки разделил по точке с запятой, к нижнему регистру привел, убрал лишние пробелы и тж. Частота - число вакансий с данным набором навыков. Условие отбора мобильных разработчиков отдельно задано в python и sql

MapReduce читает gzip по S3 частями. Отдельный процесс маппер создаёт ключи для одиночных навыков, неупорядоченных пар и троек, а reducer суммирует значения. DuckDB сам читает тот же S3 объект через httpfs, строит сочетания по индексам $i < j < k$ и группирует их в sql.

Таблица 1 - Контрольные числа двух способов расчёта **(расхождений нет)**

Навыков в наборе	Разных наборов MapReduce / SQL	Сумма частот MapReduce / SQL
1	980 / 980	14 760 / 14 760
2	11 714 / 11 714	46 585 / 46 585
3	58 848 / 58 848	123 896 / 123 896

При s разных навыках в вакансии получается s одиночных навыков, $s(s − 1)/2$ пар и $s(s − 1)(s − 2)/6$ троек. В двух вакансиях найдено по 28 навыков: каждая даёт 378 пар и 3 276 троек. Поэтому сумма частот растёт с 14 760 до 46 585 и 123 896. **Резы здесь: skills_1.csv, skills_2.csv и skills_3.csv.**

4 Верификация результатов

Сравнил все частоты, количество зарплат и медианы по сочетаниям, состав выбранных вакансий и обе региональные таблицы во всех 23 запусках. Проверка также пересчитывает суммы частот по числу сочетаний в каждой вакансии.

```
python3 scripts/analyze.py
variant=9; input_rows=229388; selected_rows=3640
with_skills=2709; salary_rows=977
MapReduce vs DuckDB: groups=133691; differences=0
k=1; distinct=980; occurrences=14760; differences=0
k=2; distinct=11714; occurrences=46585; differences=0
k=3; distinct=58848; occurrences=123896; differences=0
salary_medians_checked=133691; differences=0
mapper_exit=23; retries=1; frequency_differences=0
verified_runs=23; winner=stream; speedup=1.077
```

Для проверки отказа первый маппер завершился через os._exit(23). Родительский процесс создал новый маппер и повторно передал тот же чанк. Reducer принял результат один раз. Частоты, медианы и выбранные вакансии совпали с обычным запуском.

```
python3 scripts/profile.py --mode stream \
  --repeat 1 --inject-failure
mapper task=0: process exited 23; retry=1
```

4.1 Результаты в хранилище

Результаты для одного, двух и трёх навыков: results/variant9/ бакета hh-lake

```
python3 scripts/publish.py
s3://hh-lake/results/variant9/skills_1.csv: HTTP 200;
bytes=26374
s3://hh-lake/results/variant9/skills_2.csv: HTTP 200;
bytes=388277
s3://hh-lake/results/variant9/skills_3.csv: HTTP 200;
bytes=2433695
```

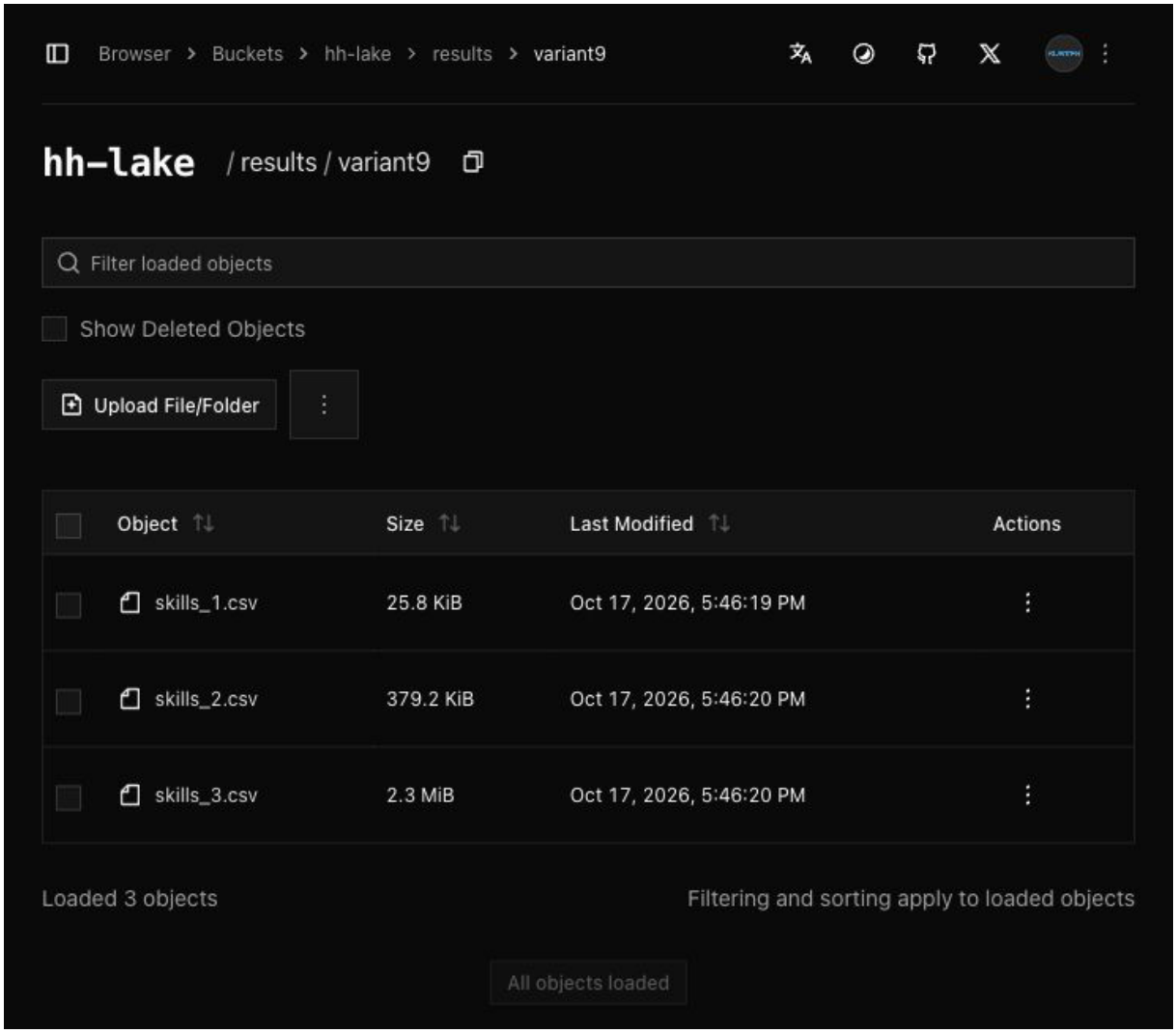


Рисунок 3 - Три файла результатов в бакете hh-lake

5 Навыки и заработная плата

Для зарплат использовал 977 вакансий с положительными значениями в RUR/RUB и gross=false, то есть получается с суммой на руки. При двух границах брал их среднее, при одной - указанную границу.

Для пары базой выбрал более частый одиночный навык, для тройки - более частую входящую пару. Частоту считал по всей выборке, медиану - по вакансиям с известной сопоставимой зарплатой. Рассматривал сочетания минимум с 20 такими вакансиями. Разница равна медиане сочетания минус медиана базы. Ниже три наибольших положительных и три наибольших отрицательных значения для каждого размера набора

Таблица 2 - Пары с наибольшим ростом и снижением медианы

Сочетание	База (частота)	N зарплат соч. / база	Медиана базы, руб.	Медиана соч., руб.	Разница руб.
ios + solid	ios (884)	26 / 302	150 000	212 500	+62 500
solid + ооп	ооп (296)	20 / 118	150 000	210 000	+60 000
solid + swift	swift (704)	20 / 181	170 000	220 000	+50 000
ios + json api	ios (884)	20 / 302	150 000	80 000	−70 000
kotlin + mvp	kotlin (1 063)	20 / 259	180 000	110 000	−70 000
java + mvvm	java (584)	23 / 168	160 000	100 000	−60 000

Для ios + solid медиана на 62 500 руб. выше базы ios: 212 500 против 150 000 руб. Для ios + json api она на 70 000 руб. ниже: 80 000 против 150 000 руб. База включает все вакансии с базовым навыком, в том числе вакансии с сочетанием.

5.1 Тройки навыков и интерпретация

Таблица 3 - Тройки с наибольшим ростом и снижением медианы

Сочетание	База (частота)	N зарплат соч. / база	Медиана базы, руб.	Медиана соч., руб.	Разница руб.
android + ios + typescript	android + ios (271)	20 / 135	135 000	190 000	+55 000
ios + swift + ооп	ios + swift (556)	40 / 144	160 000	195 000	+35 000
android sdk + git + kotlin	android sdk + kotlin (513)	46 / 118	155 000	187 500	+32 500
java + kotlin + mvvm	java + kotlin (495)	22 / 132	160 000	100 000	−60 000
android + android sdk + mvvm	android + android sdk (458)	27 / 117	160 000	110 000	−50 000
android + kotlin + ооп	android + kotlin (777)	36 / 204	180 000	140 000	−40 000

Наибольший рост у android + ios + typescript: 190 000 против 135 000 руб. у базы android + ios, разница +55 000 руб. Наибольшее снижение у java + kotlin + mvvm: 100 000 против 160 000 руб. у java + kotlin, разница −60 000 руб.

У java + kotlin + mvvm в названии 4 из 22 вакансий с зарплатой есть junior. у базы java + kotlin - 5 из 132. Большая доля джунов может объяснять снижение медианы этой тройки.

Разница медианных зарплат может быть связана с опытом сотрудников, регионом и работодателем. Поэтому она не означает, что добавление навыка само по себе повысит или понизит зарплату на указанную сумму.

6 Сравнение регионов

Выбрал Москву, Санкт-Петербург и Новосибирск. Долю набора считал от всех вакансий варианта в регионе, включая записи без навыков. Правила расчёта зарплат те же.

Таблица 4 - Вакансии и зарплаты по регионам

Регион	Вакансий	С навыками	С зарплатой	Медиана, руб.
Москва	1 691	1 216	403	200 000
Санкт-Петербург	567	402	149	160 000
Новосибирск	133	115	35	125 000

Таблица 5 - Наиболее востребованные навыки пары и тройки

Регион / навыков	Самый частый набор	Частота	Доля, %	N зарплат	Медиана, руб.
Москва / 1	kotlin	481	28,44	101	250 000
Москва / 2	android + kotlin	337	19,93	80	245 000
Москва / 3	android + android sdk + kotlin	158	9,34	35	230 000
СПб / 1	android	177	31,22	62	160 000
СПб / 2	android + kotlin	131	23,10	38	177 500
СПб / 3	android + java + kotlin	69	12,17	24	160 000
Новосибирск / 1	android	42	31,58	10	150 000
Новосибирск / 2	android + kotlin	24	18,05	7	150 000
Новосибирск / 3	android + android sdk + kotlin	10	7,52	4	175 000

Медиана Москвы на 25% выше Санкт-Петербурга и на 60% выше Новосибирска. Android + kotlin - самая частая пара во всех трёх регионах. максимальная доля в Санкт-Петербурге - 23,10%. В наиболее частой тройке Санкт-Петербурга присутствует java, Москвы и Новосибирска - android sdk. У ведущей тройки Новосибирска медиана 175000 руб, но известны только четыре зарплаты

Бонусное задание

Замерял полный запуск: чтение S3, фильтрацию, частоты, медианы, регионы и запись результатов. Пик памяти - сумма RSS родителя и дочерних процессов с опросом каждые 20 мс, без RustFS. Httpfs установлен до замеров. Запуски последовательные + ОС могла кэшировать данные.

Исходная версия использует csv.DictReader и хранит целиком чанк из 32 768 строк, маппер выдаёт каждую пару «ключ, 1». В оптимизированной версии stream фильтрую строки по мере чтения, сохраняю в чанке только вакансии своего варианта, а маппер сразу суммирует одинаковые ключи. Размер входного чанка тот же.

Таблица 6 - Время и память MapReduce и DuckDB

Способ	Время попыток, с	Пик RSS попыток, МиБ	Медиана времени, с	Медиана RSS, МиБ
MapReduce до	6,919 6,788 6,785 6,774	575,5 574,4 574,2 572,4	6,787	574,3
MapReduce после	6,301 6,299 6,288 6,326	196,7 202,1 197,5 195,9	6,300	197,1
DuckDB	4,328 4,049 4,046	490,5 487,4 491,4	4,049	490,5

Для каждого способа выполнено не менее трёх попыток + для исходной версии и stream добавил четвёртую проверочную попытку. Запуски с отказом маппера в медианы профилирования не включал.

stream: медианное время снизилось с 6,787 до 6,300 с (7,17%), память - с 574,3 до 197,1 МиБ (65,69%). Ускорение получилось 1,077 раз.

Но DuckDB быстрее, он выполняет обработку внутри движка. В mapreduce добавляются расходы на python объекты и передачу данных между процессами. Оптимизация stream уменьшает память, поскольку в чанке сохраняются только вакансии моего варианта.